

République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université Abderrahmane Mira de Béjaïa
Faculté des Sciences Exactes
Département de Recherche Opérationnelle



Création d'un Entrepôt de Données et Analyse OLAP – Cas de la vente en ligne

Présenté par :

MESROUR LOUNIS

RABIA NASSIM

MOKHTARI ANIS BADREDINE

DAHMANI MAHDI

CHAMEN RAYANE

Année Universitaire 2024 – 2025

1

Introduction Générale

Dans un contexte où les données jouent un rôle stratégique dans la prise de décision, la capacité à collecter, transformer et analyser efficacement des volumes importants d'informations constitue un atout majeur pour les organisations. Le développement des systèmes d'aide à la décision, notamment à travers les entrepôts de données et les outils d'analyse OLAP (On-Line Analytical Processing), permet d'exploiter ces données pour en extraire des connaissances utiles.

Ce projet a pour objectif de mettre en œuvre un entrepôt de données complet, depuis le prétraitement des données sources jusqu'à l'analyse multidimensionnelle à l'aide d'outils de visualisation.

Nous avons choisi le secteur du e-commerce pour ce projet, un domaine particulièrement riche en données transactionnelles et comportementales. À partir d'un jeu de données réelles, nous avons conçu une architecture décisionnelle incluant un schéma en étoile, plusieurs data marts thématiques, ainsi qu'une vue multidimensionnelle intégrée dans Power BI pour réaliser des analyses OLAP.

Ce travail nous a permis d'appliquer de manière concrète les concepts de modélisation dimensionnelle, de processus ETL (Extract, Transform, Load) et de visualisation analytique, tout en répondant à des problématiques métiers telles que l'optimisation des ventes, l'analyse du comportement client, ou encore la compréhension des tendances de consommation.

2

Données : description et nettoyage

2.1 Description du jeu de données

Le jeu de données utilisé dans ce projet provient du secteur du e-commerce. Il contient des informations détaillées sur les achats effectués par des clients. Les colonnes principales du fichier sont les suivantes :

- `identifiant_client` : identifiant unique du client,
- `date_achat` : date et heure de l'achat,
- `categorie_produit` : catégorie du produit acheté,
- `prix_produit` : prix unitaire du produit,
- `quantite` : nombre d'unités achetées,
- `montant_total_achat` : total payé pour l'achat,
- `methode_paiement` : méthode de paiement utilisée,
- `age_client` : âge du client,
- `retours` : indicateur de retour produit (0 ou 1),
- `nom_client` : nom du client,
- `age` : doublon d'âge (supprimé),
- `genre` : sexe du client,
- `taux_de_desabonnement` : probabilité que le client se désabonne.

Ces données ont été importées depuis un fichier CSV et stockées dans une base de données PostgreSQL pour les étapes suivantes de transformation et d'analyse.

2.2 Chargement initial dans PostgreSQL

Avant tout traitement, les données brutes ont été insérées dans une table appelée `achats_clients`. Voici le script SQL utilisé :

```

--- Chargement les données dans une table
CREATE TABLE achats_clients (
  identifiant_client INTEGER,
  date_achat TIMESTAMP,
  categorie_produit TEXT,
  prix_produit NUMERIC,
  quantite INTEGER,
  montant_total_achat NUMERIC,
  methode_paiement TEXT,
  age_client INTEGER,
  retours NUMERIC,
  nom_client TEXT,
  age INTEGER,
  genre TEXT,
  taux_de_desabonnement INTEGER
);

```

2.3 Prétraitement des données

Avant d'effectuer la modélisation de l'entrepôt de données, un prétraitement a été réalisé afin d'améliorer la qualité et la structure des données. Les étapes réalisées sont les suivantes :

- Ajout de trois colonnes temporelles dérivées de `date_achat` : `annee`, `mois` et `jour_semaine`.
- Remplissage de ces colonnes à l'aide de fonctions SQL d'extraction de date.
- Suppression de la colonne `age`, car elle était redondante avec `age_client`.
- Suppression des lignes où la colonne `retours` était vide (47 596 lignes supprimées).
- Vérification de l'absence de doublons exacts dans les données restantes.

Le script SQL correspondant est le suivant :

```

--- 1)Prétraitement-- Ajouter les colonnes annee, mois, jour
ALTER TABLE achats_clients ADD COLUMN annee INT;
ALTER TABLE achats_clients ADD COLUMN mois INT;
ALTER TABLE achats_clients ADD COLUMN jour_semaine TEXT;
-- Remplir les colonnes
UPDATE achats_clients
SET
  annee = EXTRACT(YEAR FROM date_achat),
  mois = EXTRACT(MONTH FROM date_achat),
  jour_semaine = TO_CHAR(date_achat, 'Day');
--- Supprimer la colonne age (puisque'elle est dupliquée avec age_client)
ALTER TABLE achats_clients DROP COLUMN age;
--- visualiser les lignes ou retours est null
SELECT *FROM achats_clients WHERE retours IS NULL
--- Supprimer les lignes où retours est NULL
DELETE FROM achats_clients WHERE retours IS NULL;--- DELETE 47596
--- détection des doublons
SELECT COUNT(*) AS nb_doublons
FROM (
  SELECT COUNT(*)
  FROM achats_clients
  GROUP BY identifiant_client, date_achat, categorie_produit, prix_produit, quantite,
  montant_total_achat, methode_paiement, age_client, retours,
  nom_client, genre,taux_de_desabonnement
  HAVING COUNT(*) > 1
) AS sous_requete;
---on a pas des doublons .

```

2.4 Conclusion

La phase de description et de nettoyage des données a permis d'aboutir à un ensemble propre, structuré et exploitable. Ce jeu de données nettoyé constitue une base solide pour la modélisation dimensionnelle et les analyses OLAP à venir.

3

Analyse ETL et Modélisation de l'entrepôt de données

3.1 Introduction

Dans cette partie, nous présentons les étapes de construction de l'entrepôt de données. Le processus suit une démarche structurée basée sur le paradigme ETL : Extraction, Transformation, Chargement. Nous détaillons ensuite la modélisation en étoile adoptée, la création des Data Marts et enfin la construction d'une vue multidimensionnelle dédiée à l'analyse via Power BI.

3.2 Processus ETL

Le processus ETL (Extract - Transform - Load) a été réalisé entièrement à l'aide de requêtes SQL exécutées sur PostgreSQL. Les étapes sont :

- **Extraction** : les données sont extraites depuis le fichier CSV `ecommerce_customer_data_custom...` et chargées dans une table brute appelée `achats_clients`.
- **Transformation** :
 - Suppression de la colonne redondante `age`,
 - Création des colonnes `annee`, `mois`, `jour_semaine`,
 - Suppression des lignes avec `retours IS NULL`,
 - Vérification de l'unicité des lignes (absence de doublons exacts).
- **Chargement** : insertion des données transformées dans les tables de dimensions et de faits selon une structure en étoile.

3.3 Modélisation en étoile

Nous avons modélisé notre entrepôt selon une architecture en étoile, avec une table centrale contenant les faits et trois tables de dimensions :

- `dim_client` : informations sur les clients,
- `dim_produit` : informations sur les produits,
- `dim_temps` : informations temporelles.

Création des tables de dimensions et de faits :

```
--- Dimension dim_client
CREATE TABLE dim_client (
    id_client SERIAL PRIMARY KEY,
    nom_client TEXT,
    age INT,
    genre TEXT,
    methode_paiement TEXT
);
---- Dimension dim_produit
CREATE TABLE dim_produit (
    id_produit SERIAL PRIMARY KEY,
    categorie_produit TEXT,
    prix_produit NUMERIC
);
---- Dimension dim_temps
CREATE TABLE dim_temps (
    id_temps SERIAL PRIMARY KEY,
    date_achat DATE,
    annee INT,
    mois INT,
    jour_semaine TEXT
);
```

FIGURE 3.1 – Création des tables de dimensions

```

',
---- Table de faits : faits_achats
CREATE TABLE faits_achats (
    id_achat SERIAL PRIMARY KEY,
    id_client INT REFERENCES dim_client(id_client),
    id_produit INT REFERENCES dim_produit(id_produit),
    id_temps INT REFERENCES dim_temps(id_temps),
    quantite INT,
    montant_total NUMERIC,
    retours NUMERIC,
    taux_de_desabonnement INT
);

```

FIGURE 3.2 – Création de la table de faits faits_achats

Insertion des données dans les dimensions :

```

--- Insérer les donnees dans dim_client
INSERT INTO dim_client (nom_client, age, genre, methode_paiement)
SELECT DISTINCT nom_client, age_client, genre, methode_paiement
FROM achats_clients;
---- INSERT 0 119004

--- Insérer les donnees dans dim_produit
INSERT INTO dim_produit (categorie_produit, prix_produit)
SELECT DISTINCT categorie_produit, prix_produit
FROM achats_clients;
--- INSERT 0 1964

--- Insérer les donnees dans dim_temps
INSERT INTO dim_temps (date_achat, annee, mois, jour_semaine)
SELECT DISTINCT DATE(date_achat), annee, mois, jour_semaine
FROM achats_clients;
--- INSERT 0 1354

```

FIGURE 3.3 – Insertion des données dans les tables de dimensions

Insertion des faits à partir des jointures :

```

--- Insérer les donnees dans faits_achats
INSERT INTO faits_achats (
    id_client, id_produit, id_temps, quantite, montant_total, retours, taux_de_desabonnement
)
SELECT
    dc.id_client,
    dp.id_produit,
    dt.id_temps,
    ac.quantite,
    ac.montant_total_achat,
    ac.retours,
    ac.taux_de_desabonnement
FROM achats_clients ac
JOIN dim_client dc
    ON ac.nom_client = dc.nom_client
    AND ac.age_client = dc.age
    AND ac.genre = dc.genre
    AND ac.methode_paiement = dc.methode_paiement
JOIN dim_produit dp
    ON ac.categorie_produit = dp.categorie_produit
    AND ac.prix_produit = dp.prix_produit
JOIN dim_temps dt
    ON DATE(ac.date_achat) = dt.date_achat
    AND ac.annee = dt.annee
    AND ac.mois = dt.mois
    AND ac.jour_semaine = dt.jour_semaine;
----- INSERT 0 202404

```

FIGURE 3.4 – Insertion dans la table faits_achats par jointure avec les dimensions

3.4 Création des Data Marts

Les Data Marts sont des vues matérialisées permettant de répondre à des besoins analytiques ciblés (produits, clients, temporalité, retours). Voici les principaux Data Marts créés :

1. Data Mart des ventes par produits :

```

---Data mart des ventes (dm_ventes_produits)
---But : Analyser les quantités vendues et les revenus par produit, catégorie, période.
CREATE MATERIALIZED VIEW dm_ventes_produits AS
SELECT
    dp.categorie_produit,
    dt.annee,
    dt.mois,
    SUM(fa.quantite) AS total_quantite,
    SUM(fa.montant_total) AS total_ventes
FROM faits_achats fa
JOIN dim_produit dp ON fa.id_produit = dp.id_produit
JOIN dim_temps dt ON fa.id_temps = dt.id_temps
GROUP BY dp.categorie_produit, dt.annee, dt.mois;

```

FIGURE 3.5 – dm_ventes_produits : Ventes et quantités par produit, mois, année

2. Data Mart du comportement des clients :


```

---Data mart client (dm_comportement_clients)
---But : Étudier le comportement des clients (retours, désabonnement, méthode de paiement)
CREATE MATERIALIZED VIEW dm_comportement_clients AS
SELECT
    dc.nom_client,
    dc.age,
    dc.genre,
    dc.methode_paiement,
    COUNT(*) AS nb_achats,
    SUM(fa.montant_total) AS total_depenses,
    AVG(fa.retours) AS taux_retour_moyen,
    AVG(fa.taux_de_desabonnement) AS taux_desabonnement_moyen
FROM faits_achats fa
JOIN dim_client dc ON fa.id_client = dc.id_client
GROUP BY dc.nom_client, dc.age, dc.genre, dc.methode_paiement;

```

FIGURE 3.6 – dm_comportement_clients : comportements d’achat, retours, désabonnements

3. Data Mart temporel :

```

---Data mart temps (dm_ventes_temps)
---But : Observer l'évolution des ventes au fil du temps (utile pour tendance et prévision).
CREATE MATERIALIZED VIEW dm_ventes_temps AS
SELECT
    dt.annee,
    dt.mois,
    dt.jour_semaine,
    SUM(fa.montant_total) AS total_ventes,
    SUM(fa.quantite) AS total_quantite
FROM faits_achats fa
JOIN dim_temps dt ON fa.id_temps = dt.id_temps
GROUP BY dt.annee, dt.mois, dt.jour_semaine;

```

FIGURE 3.7 – dm_ventes_temps : évolution des ventes par jour, mois et année

4. Data Mart des retours :

```

---Data mart des produits retournés (dm_retours_produits)
---But : Identifier les catégories avec le plus de retours.

CREATE MATERIALIZED VIEW dm_retours_produits AS
SELECT
    dp.categorie_produit,
    AVG(fa.retours) AS taux_moyen_retour,
    SUM(fa.retours) AS nb_total_retours,
    COUNT(*) AS nb_achats
FROM faits_achats fa
JOIN dim_produit dp ON fa.id_produit = dp.id_produit
GROUP BY dp.categorie_produit;

```

FIGURE 3.8 – dm_retours_produits : taux moyen de retour par catégorie de produit

3.5 Création de la Vue Multidimensionnelle

Pour l'analyse dans Power BI, nous avons créé une vue plate qui fusionne toutes les dimensions et les faits dans une seule vue matérialisée appelée `vue_multidimensionnelle`.

```
---creation dune table multidimensionnelle
CREATE MATERIALIZED VIEW vue_multidimensionnelle AS
SELECT
    fa.id_achat,
    -- Dimension client
    dc.id_client,
    dc.nom_client,
    dc.age,
    dc.genre,
    dc.methode_paiement,
    -- Dimension produit
    dp.id_produit,
    dp.categorie_produit,
    dp.prix_produit,
    -- Dimension temps
    dt.id_temps,
    dt.date_achat, dt.annee,
    dt.mois, dt.jour_semaine,
    -- Faits
    fa.quantite, fa.montant_total,
    fa.retours, fa.taux_de_desabonnement
FROM faits_achats fa
JOIN dim_client dc ON fa.id_client = dc.id_client
JOIN dim_produit dp ON fa.id_produit = dp.id_produit
JOIN dim_temps dt ON fa.id_temps = dt.id_temps;
```

FIGURE 3.9 – Vue multidimensionnelle pour Power BI

Cette vue permet un import rapide et une navigation fluide dans Power BI, facilitant les opérations OLAP telles que le drill-down, le roll-up, le slicing ou le dicing.

3.6 Conclusion

Ce chapitre a détaillé la construction technique de notre entrepôt de données. À travers les étapes ETL, la modélisation en étoile, la création des Data Marts et de la vue multidimensionnelle, nous avons mis en place une base solide pour l'analyse décisionnelle à l'aide d'outils comme Power BI.

4

Analyse OLAP

4.1 Présentation Générale

L'analyse OLAP (On-Line Analytical Processing) permet une exploration interactive et multidimensionnelle des données afin de répondre à des problématiques décisionnelles spécifiques. Grâce à la structure en étoile de notre entrepôt de données et à la vue *vue_multidimensionnelle*, nous avons pu importer l'ensemble des données nécessaires dans Power BI et créer un tableau de bord analytique complet.

4.2 Structure de la Table Multidimensionnelle

La vue *vue_multidimensionnelle* regroupe les informations issues des trois dimensions (client, produit, temps) et de la table de faits, facilitant l'intégration dans l'outil d'analyse. Voici la structure des données importées :

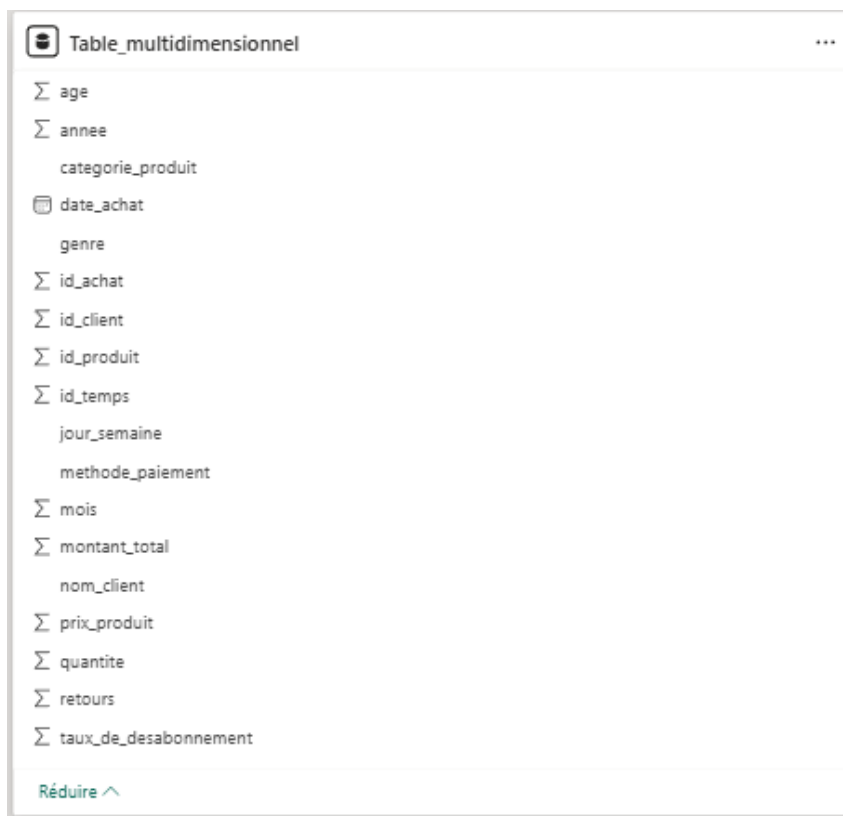


FIGURE 4.1 – Structure de la table multidimensionnelle dans Power BI

Cette structure nous a permis de construire des indicateurs de performance et de réaliser diverses opérations analytiques sur l'ensemble des axes métiers (temps, client, produit, etc.).

4.3 Tableau de Bord et Indicateurs

Nous avons construit un tableau de bord interactif sous Power BI à partir de cette vue multidimensionnelle. Il comprend plusieurs indicateurs clés de performance (KPIs), des filtres et des graphiques analytiques.

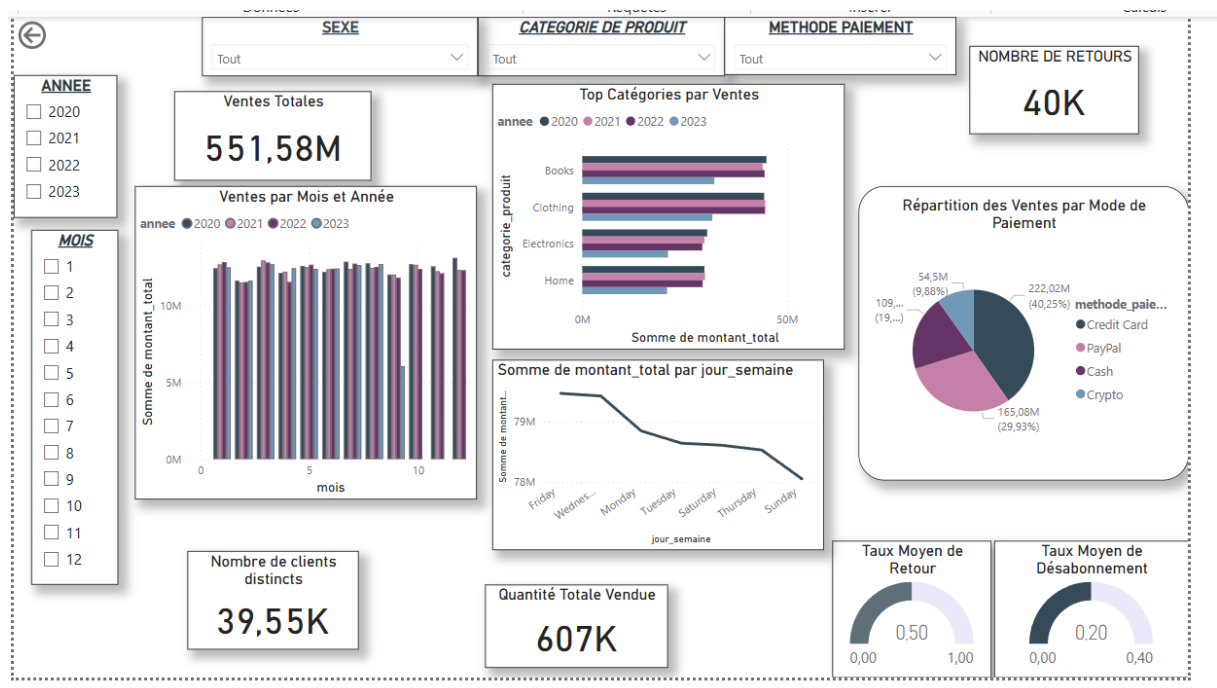


FIGURE 4.2 – Dashboard Power BI : analyse des ventes et du comportement client

Voici les éléments notables du tableau de bord :

- **Filtres dynamiques** sur le sexe, la catégorie de produit, la méthode de paiement, l'année et le mois.
- **KPI globaux** :
 - Ventes totales : 551,58M.
 - Nombre de retours : 40K.
 - Nombre de clients distincts : 39,55K.
 - Quantité totale vendue : 607K.
 - Taux moyen de retour : 0,50.
 - Taux moyen de désabonnement : 0,20.
- **Graphiques** :
 - Ventes par mois et année.
 - Top catégories par ventes.
 - Répartition des ventes par mode de paiement.
 - Évolution des ventes par jour de la semaine.

4.4 Opérations OLAP réalisées

Les opérations OLAP implémentées dans Power BI incluent :

- **Roll-up** : agrégation des ventes par mois puis par année.
- **Drill-down** : exploration des ventes par client, produit et période.
- **Slice** : sélection des ventes selon le genre ou une méthode de paiement.

- **Dice** : croisement de plusieurs dimensions (ex. : ventes par genre et par produit sur une période donnée).

4.5 Conclusion

L'analyse OLAP nous a permis d'exploiter efficacement la structure multidimensionnelle de l'entrepôt. Le tableau de bord Power BI a fourni une visualisation claire et interactive des ventes, du comportement client et des performances produits, facilitant la prise de décisions stratégiques fondées sur les données.

5

Requêtes analytiques

5.1 Introduction

Ce chapitre présente plusieurs requêtes SQL exécutées sur notre entrepôt de données. Elles ont pour but d'extraire des indicateurs utiles à l'analyse métier, tels que les produits les plus vendus, le comportement des clients ou encore la performance des méthodes de paiement.

5.2 Requête 1 – Top 5 des catégories de produits les plus vendues

Cette requête permet d'identifier les catégories de produits les plus populaires en termes de volume de vente.

```
SELECT
    dp.categorie_produit,
    SUM(fa.quantite) AS total_quantite
FROM faits_achats fa
JOIN dim_produit dp ON fa.id_produit = dp.id_produit
GROUP BY dp.categorie_produit
ORDER BY total_quantite DESC
LIMIT 5;
```

	categorie_produit text	total_quantite bigint
1	Clothing	182510
2	Books	181311
3	Electronics	122104
4	Home	121139

FIGURE 5.1 – *
Requête SQL

FIGURE 5.2 – *
Résultat SQL

FIGURE 5.3 – Analyse des ventes par catégorie de produit

5.3 Requête 2 – Taux moyen de retour par genre

Cette requête permet de comparer le comportement de retour entre les genres.

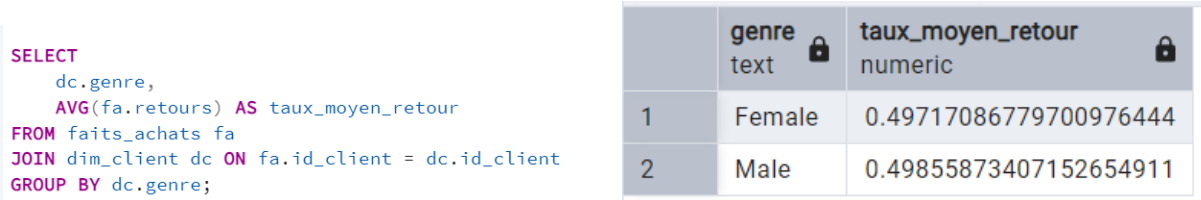


FIGURE 5.4 – *
Requête SQL

FIGURE 5.5 – *
Résultat SQL

FIGURE 5.6 – Taux de retour moyen par genre

5.4 Requête 3 – Méthode de paiement la plus rentable

Cette requête affiche le total des ventes par méthode de paiement.

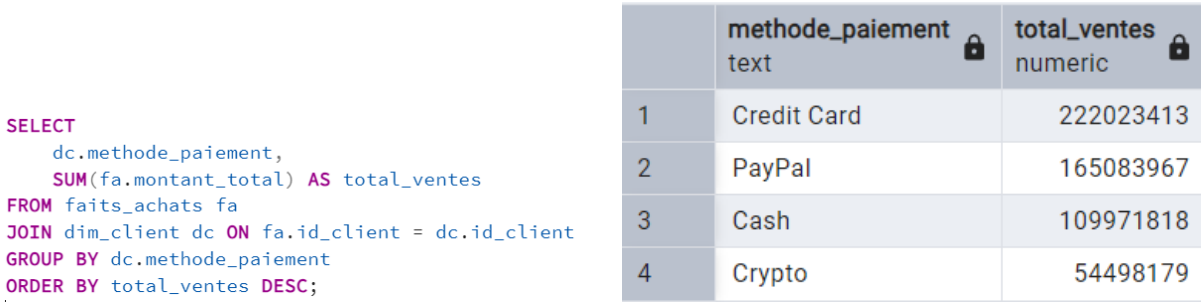


FIGURE 5.7 – *
Requête SQL

FIGURE 5.8 – *
Résultat SQL

FIGURE 5.9 – Ventes totales par méthode de paiement

5.5 Requête 4 – Répartition des ventes par jour de la semaine

Cette requête permet de visualiser les jours les plus performants en termes de chiffre d'affaires.

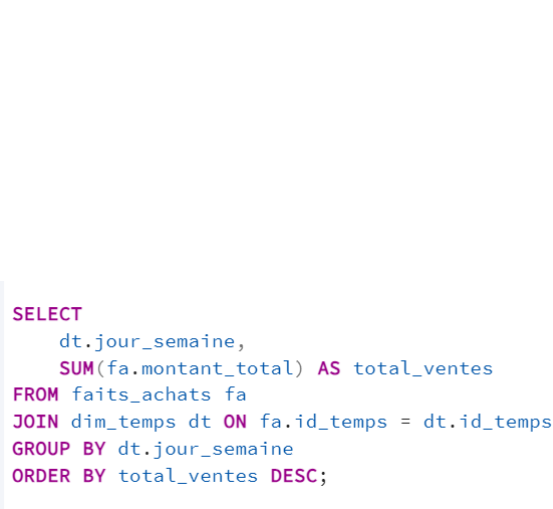


FIGURE 5.10 – *
Requête SQL

	jour_semaine text	total_ventes numeric
1	Friday	79465426
2	Wednesday	79422535
3	Monday	78847312
4	Tuesday	78644516
5	Saturday	78611097
6	Thursday	78531370
7	Sunday	78055121

FIGURE 5.11 – *
Résultat SQL

FIGURE 5.12 – Ventes par jour de la semaine

5.6 Requête 5 – Clients avec le plus d’achats

Cette requête affiche les clients ayant effectué plus de 80 achats.

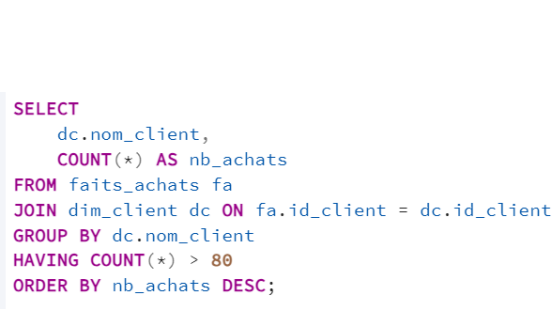


FIGURE 5.13 – *
Requête SQL

	nom_client text	nb_achats bigint
1	John Smith	91
2	Jennifer Smith	83
3	Michael Smith	82

FIGURE 5.14 – *
Résultat SQL

FIGURE 5.15 – Liste des clients les plus actifs

5.7 Conclusion

Ces analyses démontrent la richesse des informations exploitables à partir de notre entrepôt de données. Grâce aux requêtes SQL, nous avons pu répondre à différentes problématiques stratégiques de manière rapide et précise.